

Santiago de Cali, 1 de Abril del 2026

Objetivos del proyecto integrador:

Grupo 1:

1. (☀ \$) Monitoreo y denuncias ambientales

Sistema de monitoreo ciudadano y reporte de vertimientos ilegales que permita registrar en tiempo real los índices de calidad del agua del río Cali.

Nota: Este sistema se apoyará en contratos inteligentes basados en blockchain para garantizar transparencia e inmutabilidad de los datos.

2. (\$) Articulación y control de recursos financieros

Sistema para integrar y coordinar los recursos económicos de las entidades involucradas en la recuperación del río, con monitoreo y alertas automáticas frente a posibles casos de fraude, ineficiencia o mal uso presupuestal, utilizando datos de la Comuna 2.

Nota: Se implementará mediante contratos inteligentes en blockchain para asegurar trazabilidad y rendición de cuentas.

Grupo 2:

3. (☺ \$) **Gobernanza colaborativa mediante DAO**

Sistema de gobernanza ambiental basado en tecnología blockchain que permita la toma conjunta de decisiones entre instituciones, empresas y comunidades, a través de un DAO (Organización Autónoma Descentralizada) y un mecanismo de tokens para participación y responsabilidad compartida.

4. (☺) **Reducción de CO2 y generación de bonos de carbono**

Desarrollo de indicadores de mejora en la calidad del agua que puedan vincularse a la reducción de emisiones de CO2 y convertirse en oportunidades de bonos de carbono, mediante acciones como transporte sostenible, huertas urbanas, restauración de suelos y recuperación ecológica.

RESULTADO PRELIMINAR GRUPO 1

Sí. Con base en la hoja 1 del Excel y el PDF de “Objetivo 2”, el contrato puede estructurarse como un instrumento de gobernanza ambiental y financiera para el Río Cali, usando variables cuantitativas para calidad de agua y ejecución, variables cualitativas para actores y observaciones, y objetivos del contrato como núcleo programático del POMCA y de la gobernanza colaborativa. En los datos adjuntos, la hoja “Hoja 1” define para el Objetivo 2 los campos “Origen de los fondos”, “Contratista”, “Monto”, “Interventores”, “% Avance”, “Fecha de inicio”, “Fecha de terminación” y “ROI ambiental”, mientras que la hoja principal incluye variables de diagnóstico como ICA_IDEAM, OD, pH, DQO, conductividad y SST para el Río Cali.

Estructura conceptual

Desde la arquitectura y el urbanismo, este contrato se puede entender como una pieza de infraestructura institucional del río: no solo monitorea el agua, sino que articula espacio público, gestión de cuenca, participación comunitaria y control del gasto en el corredor fluvial desde el sector Zoológico de Cali o Santa Teresita hasta la desembocadura en el río Cauca, incluyendo nodos intermedios y el paso por Puerto Chontaduro como punto crítico de vigilancia y gobernanza social. Esto encaja con un enfoque POMCA porque el diagnóstico se alimenta con estaciones y variables del Excel, mientras el contrato fija reglas para intervención, seguimiento, alertas, desembolsos y evaluación socioambiental.

Los puntos del Río Cali presentes en la base muestran una lógica de entrada, tramos intermedios y salida del perímetro urbano, incluyendo referencias como Bocatoma Acueducto San Antonio, Museo La Tertulia, Clínica Los Remedios, barrio La Isla, Cartones América y la salida antes de la desembocadura al Cauca. Esa secuencia permite modelar el contrato como un corredor fluvial urbano con nodos de control, al que puedes sumar manualmente Puerto Chontaduro como punto estratégico de denuncia, participación y verificación comunitaria dentro de la gobernanza del proyecto.

POMCA contractual

El diagnóstico de cuenca debe cargar al contrato los indicadores cuantitativos base del Excel: ICA_IDEAM, OD, pH, DQO, CE y SST, asociados a puntos de muestreo y fechas, para definir estado inicial, tramos críticos y metas de mejoramiento. Los objetivos del contrato pueden formularse así: restaurar la integración fluvial del Río Cali; mejorar la calidad hídrica en el corredor urbano; asegurar trazabilidad de fondos; activar denuncia y control ciudadano; y vincular reducción de CO₂ y bonos de carbono a resultados verificables.

El plan de acción puede dividirse en cinco frentes: monitoreo sensor-comunitario, recuperación ecológica del corredor, control de vertimientos, activación urbana y pedagogía ambiental, y liberación condicionada de recursos por cumplimiento. La

participación comunitaria se traduce en reportes ciudadanos, validación territorial de incidentes, votaciones DAO y auditoría social de contratistas e interventores; y el monitoreo y evaluación se materializa con hitos, métricas ambientales, porcentaje de avance y ROI ambiental, todos ya sugeridos en el PDF del Objetivo 2.

Gobernanza y DeFi

El protocolo de gobernanza puede incorporar un esquema DeFi sencillo en el que los fondos del proyecto se depositen en el contrato y solo se liberen por tramos cuando se cumplan hitos técnicos, sociales y ambientales validados por interventores y, opcionalmente, por una DAO comunitaria. Esto responde a tu solicitud de que el contrato sea articulador y controlador de recursos financieros, y además crea trazabilidad entre origen del fondo, monto, avance de obra y resultado ambiental.

En clave IAD, las instituciones serían: autoridad ambiental, alcaldía o PDL, contratista, interventoría, comunidad ribereña, academia y DAO de seguimiento; las reglas de posición, información, agregación y pago quedarían programadas como permisos, validaciones, votaciones y desembolsos. Así, la DAO no reemplaza al Estado, sino que funciona como capa de coordinación colectiva y vigilancia distribuida sobre decisiones, reportes y ejecución de acciones en la cuenca.

Código Remix

El siguiente contrato en Solidity está diseñado para Remix IDE y organiza el modelo en tres bloques de color lógico: variables cuantitativas “verdes”, variables cualitativas “rojas” y objetivos/funciones del contrato “azules” mediante comentarios para que puedas identificarlo fácilmente en tu documento y en la explicación del código.

text

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract RioCaliPOMCA {

    address public admin;

    constructor() {
        admin = msg.sender;
    }

    modifier onlyAdmin() {
        require(msg.sender == admin, "Solo admin");
    }
}
```

```

        -;
    }

    modifier onlyAuthorized() {
        require(
            authorizedActors[msg.sender] || msg.sender ==
                admin,
            "Actor no autorizado"
        );
        -;
    }

    enum ActorType {
        Ninguno,
        Autoridad,
        Contratista,
        Interventor,
        Comunidad,
        DAO
    }

    mapping(address => bool) public authorizedActors;
    mapping(address => ActorType) public actorRoles;

    function authorizeActor(address actor, ActorType role)
        external onlyAdmin {
        authorizedActors[actor] = true;
        actorRoles[actor] = role;
    }

    function revokeActor(address actor) external onlyAdmin {
        authorizedActors[actor] = false;
        actorRoles[actor] = ActorType.Ninguno;
    }

    // =====
    // AZUL: OBJETIVOS DEL CONTRATO
    // =====

```

```

        string[] public contractObjectives;

    function addObjective(string calldata objectiveText)
        external onlyAdmin {
            contractObjectives.push(objectiveText);
        }

    function getObjective(uint256 index) external view returns
        (string memory) {
        require(index < contractObjectives.length, "Indice
            invalido");
        return contractObjectives[index];
    }

    function objectivesCount() external view returns (uint256)
    {
        return contractObjectives.length;
    }

    // =====
    // ROJO: VARIABLES CUALITATIVAS
    // =====
    struct BasinActionPlan {
        string basinDiagnosis;
        string actionPlan;
        string communityParticipation;
        string monitoringEvaluation;
        string localDevelopmentPlan;
        string environmentalComplaintStrategy;
        string governanceModel;
        string IADFramework;
        string daoProposal;
        bool exists;
    }

    BasinActionPlan public pomca;

    function setPOMCADATA(

```

```

        string calldata _basinDiagnosis,
        string calldata _actionPlan,
    string calldata _communityParticipation,
        string calldata _monitoringEvaluation,
        string calldata _localDevelopmentPlan,
    string calldata _environmentalComplaintStrategy,
        string calldata _governanceModel,
        string calldata _iadFramework,
        string calldata _daoProposal
    ) external onlyAdmin {
        pomca = BasinActionPlan({
            basinDiagnosis: _basinDiagnosis,
            actionPlan: _actionPlan,
communityParticipation: _communityParticipation,
            monitoringEvaluation: _monitoringEvaluation,
            localDevelopmentPlan: _localDevelopmentPlan,
            environmentalComplaintStrategy:
                _environmentalComplaintStrategy,
            governanceModel: _governanceModel,
            IADFramework: _iadFramework,
            daoProposal: _daoProposal,
            exists: true
        });
    }

```

```

struct FundingRecord {
    string originOfFunds;
    string contractor;
    string interventors;
    string startDate;
    string endDate;
    uint256 amount;
    uint256 progressPercent;
    uint256 environmentalROI;
    bool released;
}

```

```

FundingRecord[] public fundingRecords;

```

```

function addFundingRecord(
string calldata _originOfFunds,
    string calldata _contractor,
string calldata _interventors,
    string calldata _startDate,
    string calldata _endDate,
        uint256 _amount,
        uint256 _progressPercent,
        uint256 _environmentalROI
) external onlyAuthorized {
    fundingRecords.push(
        FundingRecord({
            originOfFunds: _originOfFunds,
            contractor: _contractor,
            interventors: _interventors,
            startDate: _startDate,
            endDate: _endDate,
            amount: _amount,
            progressPercent: _progressPercent,
            environmentalROI: _environmentalROI,
            released: false
        })
    );
}

```

```

function markFundingReleased(uint256 index) external
    onlyAdmin {
    require(index < fundingRecords.length, "Indice
        invalido");
    fundingRecords[index].released = true;
}

```

```

function fundingCount() external view returns (uint256) {
    return fundingRecords.length;
}

```

```

// =====

```

```

// VERDE: VARIABLES CUANTITATIVAS
// =====
    struct MonitoringPoint {
        string pointName;
        string sector;
        string riverSection;
        bool exists;
    }

mapping(bytes32 => MonitoringPoint) public
    monitoringPoints;
bytes32[] public monitoringPointIds;

function addMonitoringPoint(
    string calldata pointName,
    string calldata sector,
    string calldata riverSection
) external onlyAdmin returns (bytes32) {
    bytes32 pointId = keccak256(
abi.encodePacked(pointName, sector, riverSection,
        block.timestamp)
    );

    monitoringPoints[pointId] = MonitoringPoint({
        pointName: pointName,
        sector: sector,
        riverSection: riverSection,
        exists: true
    });

    monitoringPointIds.push(pointId);
    return pointId;
}

    struct WaterMeasurement {
        uint256 timestamp;
        uint256 yearValue;
        uint256 icaIDEAM;
    }

```



```

        uint256 odPercentage;
        uint256 phX100;
        uint256 dqiMgL;
        uint256 ceMicrosegCm;
        uint256 sstMgL;
        string samplingDateText;
        string qualitativeObservation;
        address reportedBy;
    }

```

```

mapping(bytes32 => WaterMeasurement[]) private
    measurementsByPoint;

```

```

event MeasurementRecorded(
    bytes32 indexed pointId,
    uint256 indexed index,
    uint256 timestamp,
    uint256 icaIDEAM,
    uint256 odPercentage,
    uint256 phX100,
    uint256 dqiMgL,
    uint256 ceMicrosegCm,
    uint256 sstMgL,
    address reportedBy
);

```

```

function addMeasurement(
    bytes32 pointId,
    uint256 _yearValue,
    uint256 _icaIDEAM,
    uint256 _odPercentage,
    uint256 _phX100,
    uint256 _dqiMgL,
    uint256 _ceMicrosegCm,
    uint256 _sstMgL,
    string calldata _samplingDateText,
    string calldata _qualitativeObservation
) external onlyAuthorized {

```

```
require(monitoredPoints[pointId].exists, "Punto no  
existe");
```

```
measurementsByPoint[pointId].push(  
    WaterMeasurement({  
        timestamp: block.timestamp,  
        yearValue: _yearValue,  
        icaIDEAM: _icaIDEAM,  
        odPercentage: _odPercentage,  
        phX100: _phX100,  
        dcoMgL: _dcoMgL,  
        ceMicrosegCm: _ceMicrosegCm,  
        sstMgL: _sstMgL,  
        samplingDateText: _samplingDateText,  
        qualitativeObservation:  
            _qualitativeObservation,  
        reportedBy: msg.sender  
    })  
);
```

```
uint256 idx = measurementsByPoint[pointId].length - 1;
```

```
emit MeasurementRecorded(  
    pointId,  
    idx,  
    block.timestamp,  
    _icaIDEAM,  
    _odPercentage,  
    _phX100,  
    _dcoMgL,  
    _ceMicrosegCm,  
    _sstMgL,  
    msg.sender  
);  
}
```

```
function measurementCount(bytes32 pointId) external view  
    returns (uint256) {
```

```

        return measurementsByPoint[pointId].length;
    }

function getMeasurement(bytes32 pointId, uint256 index)
    external
    view
    returns (
        uint256 timestamp,
        uint256 yearValue,
        uint256 icaIDEAM,
        uint256 odPercentage,
        uint256 phX100,
        uint256 dqiMgL,
        uint256 ceMicrosegCm,
        uint256 sstMgL,
        string memory samplingDateText,
        string memory qualitativeObservation,
        address reportedBy
    )
    {
require(index < measurementsByPoint[pointId].length,
    "Indice invalido");
        WaterMeasurement memory m =
        measurementsByPoint[pointId][index];
        return (
            m.timestamp,
            m.yearValue,
            m.icaIDEAM,
            m.odPercentage,
            m.phX100,
            m.dqiMgL,
            m.ceMicrosegCm,
            m.sstMgL,
            m.samplingDateText,
            m.qualitativeObservation,
            m.reportedBy
        );
    }

```

```

// =====
// DAO Y DENUNCIAS
// =====
    struct Proposal {
        string title;
        string description;
        uint256 yesVotes;
        uint256 noVotes;
        uint256 deadline;
        bool executed;
    }

    Proposal[] public proposals;
mapping(uint256 => mapping(address => bool)) public
    hasVoted;

    function createProposal(
        string calldata title,
        string calldata description,
        uint256 durationSeconds
    ) external onlyAuthorized {
        require(
            actorRoles[msg.sender] == ActorType.DAO ||
            actorRoles[msg.sender] == ActorType.Autoridad ||
            msg.sender == admin,
            "No puede crear propuestas"
        );

        proposals.push(
            Proposal({
                title: title,
                description: description,
                yesVotes: 0,
                noVotes: 0,
                deadline: block.timestamp + durationSeconds,
                executed: false
            })
        )
    }

```

```

        );
    }

function voteProposal(uint256 proposalId, bool support)
    external onlyAuthorized {
        require(proposalId < proposals.length, "Propuesta
            invalida");
        require(block.timestamp <=
proposals[proposalId].deadline, "Votacion cerrada");
        require(!hasVoted[proposalId][msg.sender], "Ya voto");

        hasVoted[proposalId][msg.sender] = true;

        if (support) {
            proposals[proposalId].yesVotes += 1;
        } else {
            proposals[proposalId].noVotes += 1;
        }
    }

function executeProposal(uint256 proposalId) external
    onlyAdmin {
        require(proposalId < proposals.length, "Propuesta
            invalida");
        Proposal storage p = proposals[proposalId];
        require(block.timestamp > p.deadline, "Votacion
            abierta");
        require(!p.executed, "Ya ejecutada");
        require(p.yesVotes > p.noVotes, "No aprobada");

        p.executed = true;
    }

function proposalsCount() external view returns (uint256)
    {
        return proposals.length;
    }

```

```

    struct EnvironmentalComplaint {
        string complaintText;
        string locationText;
        uint256 timestamp;
        address reporter;
    }

    EnvironmentalComplaint[] public complaints;

    function fileComplaint(
        string calldata complaintText,
        string calldata locationText
    ) external onlyAuthorized {
        complaints.push(
            EnvironmentalComplaint({
                complaintText: complaintText,
                locationText: locationText,
                timestamp: block.timestamp,
                reporter: msg.sender
            })
        );
    }

    function complaintsCount() external view returns (uint256)
    {
        return complaints.length;
    }

    // =====
    // CO2 Y BONOS DE CARBONO
    // =====
    struct CarbonRecord {
        uint256 co2ReducedKg;
        uint256 carbonCredits;
        string methodology;
        string evidence;
        uint256 timestamp;
    }

```

```

CarbonRecord[] public carbonRecords;

function addCarbonRecord(
    uint256 _co2ReducedKg,
    uint256 _carbonCredits,
    string calldata _methodology,
    string calldata _evidence
) external onlyAuthorized {
    carbonRecords.push(
        CarbonRecord({
            co2ReducedKg: _co2ReducedKg,
            carbonCredits: _carbonCredits,
            methodology: _methodology,
            evidence: _evidence,
            timestamp: block.timestamp
        })
    );
}

function carbonRecordsCount() external view returns
    (uint256) {
    return carbonRecords.length;
}
}

```

RESULTADO PRELIMINAR GRUPO 2

Gobernanza de la DAO “Río Cali 2050”

Blockchain, recicladores, comunidad y ONG La Papaya

1. Misión de la DAO Río Cali 2050: Coordinar, financiar y monitorear acciones para descontaminar el Río Cali y fortalecer el tejido social de las comunidades ribereñas (barrio La Isla), a través de un modelo de gobernanza comunitaria habilitado por blockchain, responsable y transparente para todos los aspectos de la comunidad (transparencia y trazabilidad, participación abierta).

Objetivos principales:

- Ambiental: Minimizar las cargas contaminantes, aumentar la calidad del agua, restauración de hábitats.

- Social: fortalecer la organización comunitaria, fomentar la creación de empleos verdes locales, educación ambiental y la participación activa de recicladores y residentes en campañas de recuperación del río dentro del barrio La Isla.
- Institucional: conectar la comunidad, universidades (incluyendo la Universidad Icesi y estudiantes de segundo semestre de la Maestría en Sostenibilidad), ONG — con La Papaya como organización de apoyo — corporaciones, agencias ambientales y el alcalde.
- Tecnológico: Blockchain, contratos inteligentes y datos abiertos como infraestructura transversal para la toma de decisiones auditable, financiamiento transparente y ejecución automatizada.

2. Estructura de miembros y actores de la DAO

- Miembros de la comunidad:
Residentes del barrio La Isla y otros barrios ribereños.
- Recicladores locales y gestores ambientales:
fundamentales en campañas de limpieza del río, incluyendo campañas de reciclaje, con incentivos por material recuperado.
- Aliados técnicos:
Universidades (como la Universidad Icesi y estudiantes de la Maestría en Sostenibilidad), grupos de investigación y científicos.
- Organizaciones sociales y ONG: principalmente con un enfoque en La Papaya, que se encargan de la articulación territorial y la educación, así como del apoyo a las comunidades.
- Sector privado:
Empresas que proporcionan recursos, instalaciones tecnológicas o infraestructura, y brindan apoyo para la producción sostenible.
- Instituciones públicas:
Administración municipal, CVC/otra autoridad ambiental y empresas de servicios públicos.

A cada miembro verificado se le otorga un token de participación, que permite:

- Proponer proyectos.
- Votar en decisiones.
- Acceder a informes y datos de monitoreo.

Órganos funcionales (en cadena + fuera de cadena)

- Asamblea de la DAO: El espacio clave de gobernanza; vota sobre propuestas clave a través de la blockchain.
- Círculos de trabajo:
- Círculo Ambiental: calidad del agua, recuperación, restauración, monitoreo.
- Círculo Social: educación, cultura, seguridad, cohesión comunitaria.
- Círculo Financiero: tesorería, alianzas, convocatorias.
- Círculo Tecnológico: tecnología blockchain, sensores, datos y paneles de control.
- Comité de Validación: un grupo mixto (comunidad + academia + autoridad) que puede revisar criterios mínimos antes de la votación en cadena.

Gobernanza comunitaria (Eje 1 de la DAO – El Corazón: DAO – Gobernar es el núcleo de la DAO)

la gobernanza es el proceso por el cual se producen decisiones colaborativas.

Establecimiento de la DAO Río Cali 2050

- Documento base.
- Implementación del contrato inteligente base.
- Registro de miembros, emisión de tokens, módulo de propuestas, votación y tesorería.

Reglas para la participación y votación

- 1 persona = 1 identidad verificada.
- Token de participación: 1 token = 1 voto o voto ponderado.
- Quórum mínimo: 51%
- Tipos de propuestas: ambientales, sociales, presupuestarias, regulatorias.

Flujo de decisiones

1. Propuesta.
2. Revisión técnica/comunitaria.
3. Votación en cadena.
4. Ejecución automática por hitos.

Financiamiento transparente

Fuentes de recursos:

- Empresas privadas.
- Fondos de cooperación y ONG.
- Contribuciones ciudadanas.
- Potencial token ambiental: RíoCaliToken.
 - Tenemos ya los presupuestos participativos, podemos partir de un supuesto de aportes de 1000 millones de pesos en empresas privadas, 1000 millones de pesos en fondos de cooperación internacional y 500 millones de contribuciones ciudadanas.

VENTA DE BONOS DE CARBONO

Se estima que una tonelada de co2 en los mercados informales se puede vender en 5 a 8 usd y en los mercados regulados entre 20 a 30 usd por tonelada. Inicialmente lapapaya estimó que toda la ruta podría capturar 600.000 ton de co2 que equivale a 3 millones de dólares por año en los mercados no regulados y 12 millones de usd año en mercados certificados, este también es un recurso adicional que se puede incorporar.

Tesorería en blockchain:

- Billetera pública.
- Contratos inteligentes de tesorería.
- Panel de transparencia.

Integración social dentro del barrio La Isla:

El corazón territorial del proyecto.

- Participación prioritaria: cuotas mínimas de tokens para residentes de los barrios.
- Talleres presenciales: aprendizaje sobre alfabetización digital y responsabilidad ambiental.

- Estrategias híbridas: asambleas presenciales y votación asistida en línea.

Tipos de proyectos:

- Infraestructura verde.
- Empleo verde.
- Apropiación de la cultura y el río.

Programas de recompensas por reciclaje

- Recicladores y residentes de La Isla se unen a programas de limpieza y recuperación del río.
- El material reciclado (plástico, vidrio, metales, etc.) se reporta en una blockchain de trazabilidad simple.
- Se proporcionan incentivos a los participantes por cada cantidad de material recuperado, incluyendo:
 - Tokens de la DAO.
 - Vales de comida.
 - Kits ambientales.
 - Créditos para capacitación o actividades comunitarias.
- La ONG La Papaya coordina la logística, pedagogía y apoyo territorial en la realización de estas campañas.

Monitoreo y control de datos ambientales

- Sensores básicos.
- Monitoreo biológico.
- Informes comunitarios.

Vinculación con blockchain.

- Oráculos de datos.
- Contratos inteligentes condicionales.
- Indicadores públicos.

Hoja de ruta básica para “Río Cali 2050”

Fase 1 – Diseño social y legal (0–6 meses)

- Proveer principios, los actores clave y reglas estándar.
- Talleres convocados con la comunidad de La Isla, con la Universidad Icesi (incluyendo a los estudiantes de la Maestría en Sostenibilidad), con La Papaya, ONG, y con autoridades.
- Redactar estatutos y arquitectura de tokens.

Fase 2: Prototipo mínimo viable de la DAO (6-12 meses)

- Despliegue para la DAO.
- Emisión piloto de tokens.
- Primeras campañas de reciclaje con incentivos.
- Financiar 2–3 proyectos iniciales

Fase 3—Escalamiento y sofisticación (12–36 meses)

- Incorporar aún más barrios y actores.
- Incluir sensores, paneles de control y oráculos.
- Escalar esfuerzos hacia campañas de reciclaje y restauración.
- Explorar un token ambiental conectado a los resultados

Comentarios generales: hasta aquí muy bien! Los felicito, logramos lo más importante que es despertar el interés, ahora es importante que cada cosa vaya en su lugar y podamos sacar un primer piloto viable y escalable, vamos muy bien!

Un CRAM (DAO Climático "Río Cali 2050 – CO₂ y Créditos de Carbono" Integrado con Monitoreo del Río Cali, Blockchain y Operación Territorial por la ONG La Papaya.

Propósito General del Sistema.

El DAO climático "Río Cali 2050 – CO₂ y Créditos de Carbono" es una infraestructura social, ambiental y tecnológica basada en blockchain para la reducción de emisiones de carbono que tiene como objetivo:

- Reducir la contaminación en el Río Cali.
- Mejorar la calidad del agua y restaurar los ecosistemas ribereños.
- Medir y convertir las mejoras ambientales en reducciones equivalentes de CO₂ (CO₂e).
- Emitir el token RCC (créditos de carbono comunitarios).
- Fortalecer el tejido social del barrio La Isla.
- Asegurar la transparencia, trazabilidad y gobernanza comunitaria.

El sistema es operado territorialmente por la ONG La Papaya y ha sido desarrollado con la ayuda técnica de estudiantes de segundo semestre de la Universidad Icesi, bajo la Maestría en Sostenibilidad.

Arquitectura DAO.

Miembros del DAO.

- Residentes del barrio La Isla.
- Recicladores locales.
- ONG La Papaya (operador principal).
- Estudiantes de la Maestría en Sostenibilidad (oráculos técnicos).
- Universidad Icesi (aliado académico).
- Sector privado (financiamiento, tecnología).
- Instituciones públicas (CVC, Alcaldía, EMCALI).

Cada miembro verificado recibe un token de participación que incluye:

- Proponer proyectos.
- Votar en decisiones.
- Tener acceso a datos ambientales y climáticos.

Organismos Funcionales.

- Asamblea DAO: votación en blockchain.
- Círculo Ambiental: monitoreo del río, restauración ecológica.
- Círculo Social: educación, cultura, cohesión comunitaria.
- Círculo Financiero: tesorería, asociaciones, créditos de carbono.
- Círculo Tecnológico: blockchain, sensores, oráculos.
- Comité de Validación:
- ONG La Papaya.
- Estudiantes de Maestría.
- Comunidad.
- Aliados Técnicos.

Gobernanza en Blockchain.

- 1 persona = 1 identidad verificada.
- 1 token = 1 voto (o ponderado para favorecer a la comunidad local).
- Quórum mínimo: 51%.
- Propuestas: ambientales; sociales; presupuestarias; regulatorias.
- Ejecución automática: contratos inteligentes liberan fondos por hitos.

Monitoreo Ambiental del Río Cali y sus Alrededores.

DAO propone un Módulo de Monitoreo Ambiental con variables, umbrales y oráculos.

Indicadores de Agua del Río Cali.

Tabla 1 – Indicadores del agua del río Cali

Variable Bueno Malo Mayor/Menor

DBO (mg/L) < 5 > 10 Menor

DQO (mg/L) < 20 > 50 Menor

SST (mg/L) < 25 > 100 Menor

Turbidez (NTU) < 5 > 50 Menor

Conductividad (µS/cm) < 300 > 700 Menor

Oxígeno disuelto (mg/L) > 6 < 4 Mayor

pH 6.5–8.5 < 6 o > 9 Rango

Temperatura (°C) 18–25 > 28 Rango

Nitrógeno total (mg/L) < 1 > 3 Menor

Fósforo total (mg/L) < 0.1 > 0.3 Menor

Índice biótico > 80 < 40 Mayor

Indicadores de Restauración del Suelo y Ecológica.

Variable Bueno Malo Mayor/Menor
Materia orgánica (%) > 5% < 2% Mayor
Cobertura vegetal (%) > 70% < 40% Mayor
Densidad de árboles (ind/ha) > 400 < 150 Mayor
Superficie revegetada (m²) > 500 < 100 Mayor
Indicadores de Movilidad Sostenible.
Variable Bueno Malo Mayor/Menor
Viajes motorizados evitados/día > 20 < 5 Mayor
Km evitados/día > 50 < 10 Mayor
Uso de bici/caminata (%) > 40% < 10% Mayor
Indicadores de Residuos y Reciclaje.
Variable Bueno Malo Mayor/Menor
Kg reciclados/mes > 500 < 100 Mayor
Kg compostados/mes > 200 < 50 Mayor
Tasa de reciclaje (%) > 30% < 10% Mayor

Oráculos Ambientales.

Los oráculos son cuentas autorizadas para almacenar datos en la blockchain:

Oráculos Principales:

ONG La Papaya:

- Operador Territorial.
- Verifica acciones ambientales realizadas.
- Registra datos sobre reciclaje, jardines, restauración.

Estudiantes de la Maestría en Sostenibilidad (Icesi):

Oráculos Técnicos.

Registran indicadores ambientales validados.

Aplican metodologías de medición.

Aliados Técnicos:

Laboratorios, sensores, grupos de investigación.

Funciones en la Cadena

registrarIndicadorAmbiental(tipo, nombre, valor, unidad)

registrarReciclaje(reciclador, kilos)

registrarCO2e(fuente, co2e, beneficiario)

Cálculo de CO₂e.

Para lograrlo, el cálculo se realiza fuera de la cadena utilizando métodos del IPCC y se registra en una blockchain.

Factores de Conversión (ejemplos).

Reciclaje:

- Plástico: 1.5–3 kg CO₂e/kg.
- Metal: 4–10 kg CO₂e/kg.
- Papel: 1–2 kg CO₂e/kg.

Movilidad:

- Coche: 0.25 kg CO₂e/km.
- Motocicleta: 0.09 kg CO₂e/km.
- Autobús: 0.05 kg CO₂e/km.

Vegetación y Suelos:

- Árbol joven: 5–10 kg CO₂e/año.
- Árbol adulto: 20–30 kg CO₂e/año.
- Jardín urbano: 1–3 kg CO₂e/m²/año.
- +1% materia orgánica = 5 t CO₂e/ha.

Agua:

- Reducción de DBO y nutrientes → menos metano → CO₂e evitado.

Créditos de Carbono Comunitarios (token RCC).

Token climático: RioCaliCarbon (RCC).

- 1 RCC = 1 kg CO₂e reducido o capturado.
- Emitido solo cuando:
- La Papaya valida la acción.
- Los estudiantes validan la medición.
- El Comité de Validación aprueba.
- El climateAdmin registra el CO₂e.

Beneficiarios:

- Recicladores.
- Residentes del barrio La Isla.
- Proyectos ambientales comunitarios;
- jardines, brigadas, restauración ecológica.

Cadena Completa del Sistema.

Acción Ambiental.

- Reciclaje, jardines, árboles, movilidad sostenible, restauración.

Medición.

- Agua, suelos, movilidad, residuos.

3. Registro en la Cadena (oráculos).

- Indicadores ambientales.

4. Cálculo de CO₂e (fuera de la cadena).

- Metodología IPCC.

Validación (La Papaya + estudiantes + Comité).

- CO₂e es aprobado.

Registro de CO₂e en Blockchain.

- registrarCO₂e(...)
- Emisión de RCC.
- Créditos de carbono comunitarios.

Uso de RCC.

- Incentivos.
- Compensación ambiental.
- Financiamiento de proyectos.

CONTRATO INTELIGENTE OBJETIVO 3

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;

/*

DAO "Río Cali 2050"

Gobernanza comunitaria en blockchain con:

- Comunidad (barrio La Isla)
- Recicladores
- ONG La Papaya
- Tesorería transparente
- Propuestas y votación con quórum mínimo del 51%
- Incentivos por reciclaje

Misión:

Coordinar, financiar y monitorear acciones para descontaminar el Río Cali y fortalecer el tejido social de las comunidades ribereñas, mediante gobernanza comunitaria habilitada por blockchain.

*/

```
contract RioCali2050DAO {
```

```
    // -----
```

```
    // ROLES PRINCIPALES
```

```
    // -----
```

```
    address public admin; // Administrador del contrato
```

```
    address public laPapaya; // ONG La Papaya (operadora de reciclaje)
```

```
    constructor(address _laPapaya) {
```

```
        admin = msg.sender;
```

```
        laPapaya = _laPapaya;
```

```
    }
```

```
    modifier soloAdmin() {
```

```
        require(msg.sender == admin, "Solo admin");
```

```
        _;
```

```
    }
```

```
    modifier soloLaPapaya() {
```

```
        require(msg.sender == laPapaya, "Solo ONG La Papaya");
```

```
        _;
```

```
    }
```

```
    // -----
```

```
    // MIEMBROS Y TOKENS DE PARTICIPACION
```

```
    // -----
```

```
    mapping(address => bool) public esMiembro;
```

```
    mapping(address => bool) public puedeVotar;
```

```
    mapping(address => uint256) public saldoTokens;
```

```
    uint256 public suministroTotal;
```

```
    event MiembroRegistrado(address miembro);
```

```
event MiembroRevocado(address miembro);
event DerechoVotoActualizado(address miembro, bool estado);
event TokensEmitidos(address destinatario, uint256 cantidad);
```

```
function registrarMiembro(
    address _miembro,
    uint256 _tokensIniciales,
    bool _puedeVotar
) external soloAdmin {
    require(!esMiembro[_miembro], "Ya es miembro");
    esMiembro[_miembro] = true;
    puedeVotar[_miembro] = _puedeVotar;
    _emitirTokens(_miembro, _tokensIniciales);
    emit MiembroRegistrado(_miembro);
}
```

```
function revocarMiembro(address _miembro) external soloAdmin {
    require(esMiembro[_miembro], "No es miembro");
    esMiembro[_miembro] = false;
    puedeVotar[_miembro] = false;
    emit MiembroRevocado(_miembro);
}
```

```
function actualizarDerechoVoto(address _miembro, bool _estado) external soloAdmin {
    require(esMiembro[_miembro], "No es miembro");
    puedeVotar[_miembro] = _estado;
    emit DerechoVotoActualizado(_miembro, _estado);
}
```

```
function _emitirTokens(address _destinatario, uint256 _cantidad) internal {
    saldoTokens[_destinatario] += _cantidad;
    suministroTotal += _cantidad;
    emit TokensEmitidos(_destinatario, _cantidad);
}
```

```
// -----
// TESORERIA EN BLOCKCHAIN
// -----
```

```
event Deposito(address remitente, uint256 monto);
event FondosEnviados(address destinatario, uint256 monto);
```

```
receive() external payable {
    emit Deposito(msg.sender, msg.value);
}
```

```
function saldoTesoreria() public view returns (uint256) {
    return address(this).balance;
}
```



```

}

// -----
// PROPUESTAS Y VOTACION
// -----

struct Propuesta {
    uint256 id;
    address proponente;
    string descripcion;
    uint256 inicio;
    uint256 fin;
    uint256 votosAFavor;
    uint256 votosEnContra;
    bool ejecutada;
    bool cancelada;
    address payable billeteraProyecto;
    uint256 montoSolicitado;
}

uint256 public contadorPropuestas;
mapping(uint256 => Propuesta) public propuestas;
mapping(uint256 => mapping(address => bool)) public yaVoto;

event PropuestaCreada(
    uint256 id,
    address proponente,
    string descripcion,
    address billeteraProyecto,
    uint256 montoSolicitado,
    uint256 inicio,
    uint256 fin
);

event VotoEmitido(uint256 idPropuesta, address votante, bool aFavor, uint256 peso);
event PropuestaEjecutada(uint256 idPropuesta, address billeteraProyecto, uint256
monto);
event PropuestaCancelada(uint256 idPropuesta);

function crearPropuesta(
    string calldata _descripcion,
    address payable _billeteraProyecto,
    uint256 _montoSolicitado,
    uint256 _duracionSegundos
) external {
    require(esMiembro[msg.sender], "Solo miembros pueden proponer");
    require(_duracionSegundos > 0, "Duracion invalida");

```

```

contadorPropuestas++;
uint256 id = contadorPropuestas;

propuestas[id] = Propuesta({
    id: id,
    proponente: msg.sender,
    descripcion: _descripcion,
    inicio: block.timestamp,
    fin: block.timestamp + _duracionSegundos,
    votosAFavor: 0,
    votosEnContra: 0,
    ejecutada: false,
    cancelada: false,
    billeteraProyecto: _billeteraProyecto,
    montoSolicitado: _montoSolicitado
});

emit PropuestaCreada(
    id,
    msg.sender,
    _descripcion,
    _billeteraProyecto,
    _montoSolicitado,
    block.timestamp,
    block.timestamp + _duracionSegundos
);
}

function votar(uint256 _idPropuesta, bool _aFavor) external {
    Propuesta storage p = propuestas[_idPropuesta];
    require(p.inicio > 0, "Propuesta inexistente");
    require(puedeVotar[msg.sender], "No tiene derecho a voto");
    require(block.timestamp >= p.inicio, "Votacion no ha iniciado");
    require(block.timestamp <= p.fin, "Votacion finalizada");
    require(!yaVoto[_idPropuesta][msg.sender], "Ya voto");
    require(!p.cancelada, "Propuesta cancelada");

    uint256 peso = saldoTokens[msg.sender];
    require(peso > 0, "Sin tokens de gobernanza");

    yaVoto[_idPropuesta][msg.sender] = true;

    if (_aFavor) {
        p.votosAFavor += peso;
    } else {
        p.votosEnContra += peso;
    }
}

```

```

        emit VotoEmitido(_idPropuesta, msg.sender, _aFavor, peso);
    }

    // -----
    // QUORUM 51% Y APROBACION
    // -----

    function poderTotalVoto() public view returns (uint256) {
        return suministroTotal;
    }

    function tieneQuorum(uint256 _idPropuesta) public view returns (bool) {
        Propuesta storage p = propuestas[_idPropuesta];
        uint256 votosTotales = p.votosAFavor + p.votosEnContra;
        uint256 quorumRequerido = (poderTotalVoto() * 51) / 100;
        return votosTotales >= quorumRequerido;
    }

    function estaAprobada(uint256 _idPropuesta) public view returns (bool) {
        Propuesta storage p = propuestas[_idPropuesta];
        if (!tieneQuorum(_idPropuesta)) return false;
        return p.votosAFavor > p.votosEnContra;
    }

    function ejecutarPropuesta(uint256 _idPropuesta) external {
        Propuesta storage p = propuestas[_idPropuesta];
        require(!p.ejecutada, "Ya ejecutada");
        require(!p.cancelada, "Propuesta cancelada");
        require(block.timestamp > p.fin, "Votacion aun en curso");
        require(estaAprobada(_idPropuesta), "No aprobada o sin quorum");
        require(address(this).balance >= p.montoSolicitado, "Fondos insuficientes");

        p.ejecutada = true;
        (bool exito, ) = p.billeteraProyecto.call{value: p.montoSolicitado}("");
        require(exito, "Error al enviar fondos");

        emit PropuestaEjecutada(_idPropuesta, p.billeteraProyecto, p.montoSolicitado);
    }

    function cancelarPropuesta(uint256 _idPropuesta) external soloAdmin {
        Propuesta storage p = propuestas[_idPropuesta];
        require(!p.ejecutada, "Ya ejecutada");
        p.cancelada = true;
        emit PropuestaCancelada(_idPropuesta);
    }

    // -----
    // PROGRAMA DE RECICLAJE E INCENTIVOS

```

```

// -----

struct RegistroReciclaje {
    uint256 id;
    address reciclador;
    uint256 kilos;
    uint256 timestamp;
    uint256 tokensIncentivo;
}

uint256 public contadorReciclaje;
mapping(uint256 => RegistroReciclaje) public registrosReciclaje;

event ReciclajeRegistrado(
    uint256 id,
    address reciclador,
    uint256 kilos,
    uint256 tokensIncentivo
);

uint256 public factorIncentivo = 10;

function actualizarFactorIncentivo(uint256 _nuevoFactor) external soloAdmin {
    factorIncentivo = _nuevoFactor;
}

function registrarReciclaje(address _reciclador, uint256 _kilos) external soloLaPapaya {
    require(_reciclador != address(0), "Reciclador invalido");
    require(_kilos > 0, "Cantidad invalida");

    contadorReciclaje++;
    uint256 id = contadorReciclaje;

    uint256 tokens = _kilos * factorIncentivo;
    _emitirTokens(_reciclador, tokens);

    registrosReciclaje[id] = RegistroReciclaje({
        id: id,
        reciclador: _reciclador,
        kilos: _kilos,
        timestamp: block.timestamp,
        tokensIncentivo: tokens
    });

    emit ReciclajeRegistrado(id, _reciclador, _kilos, tokens);
}

// -----

```

```
// UTILIDADES
// -----

function cambiarLaPapaya(address _nuevaLaPapaya) external soloAdmin {
    laPapaya = _nuevaLaPapaya;
}

function transferirAdmin(address _nuevoAdmin) external soloAdmin {
    require(_nuevoAdmin != address(0), "Admin invalido");
    admin = _nuevoAdmin;
}
}
```

CONTRATO OBJETIVO 4 EMISION DE BONOS DE CARBONO

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```
/*
DAO climático “Río Cali 2050 – CO2 y Bonos de Carbono”
Operado territorialmente por la ONG La Papaya
```

Integra:

- Gobernanza comunitaria (barrio La Isla, recicladores, aliados)
- Tokens de participación (gobernanza)
- Propuestas y votación (quórum mínimo 51%)
- Tesorería en blockchain
- Programa de reciclaje con incentivos (operado por La Papaya)
- Monitoreo ambiental del río Cali y su entorno (oráculos)
- Registro de CO₂e y emisión de bonos de carbono comunitarios (RCC)
- Rol central de la ONG La Papaya como:
 - * Operador territorial
 - * Oráculo ambiental
 - * Validador de acciones comunitarias

```
*/
```

```
contract RioCali2050ClimateDAO {
```

```
// -----
```

```
// ROLES PRINCIPALES
```

```
// -----
```

```
address public admin; // Admin general del contrato
```

```
address public laPapaya; // ONG La Papaya: operador territorial principal
```

```
address public climateAdmin; // Rol para registrar CO2e y disparar emisión de RCC
```

```
// Oráculos ambientales: La Papaya, estudiantes Maestría, aliados técnicos
mapping(address => bool) public esOraculoAmbiental;
```

```

constructor(address _laPapaya, address _climateAdmin) {
    admin = msg.sender;
    laPapaya = _laPapaya;
    climateAdmin = _climateAdmin;

    // La Papaya y climateAdmin son oráculos por defecto
    esOraculoAmbiental[_laPapaya] = true;
    esOraculoAmbiental[_climateAdmin] = true;
}

modifier soloAdmin() {
    require(msg.sender == admin, "Solo admin");
    _;
}

modifier soloLaPapaya() {
    require(msg.sender == laPapaya, "Solo ONG La Papaya");
    _;
}

modifier soloOraculoAmbiental() {
    require(esOraculoAmbiental[msg.sender], "No es oraculo ambiental");
    _;
}

modifier soloClimateAdmin() {
    require(msg.sender == climateAdmin, "Solo climateAdmin");
    _;
}

// -----
// MIEMBROS Y TOKENS DE GOBERNANZA
// -----

mapping(address => bool) public esMiembro;
mapping(address => bool) public puedeVotar;

mapping(address => uint256) public saldoTokensGob;
uint256 public suministroTotalGob;

event MiembroRegistrado(address miembro);
event MiembroRevocado(address miembro);
event DerechoVotoActualizado(address miembro, bool estado);
event TokensGobEmitidos(address destinatario, uint256 cantidad);

function registrarMiembro(
    address _miembro,

```

```

        uint256 _tokensIniciales,
        bool _puedeVotar
    ) external soloAdmin {
        require(!esMiembro[_miembro], "Ya es miembro");
        esMiembro[_miembro] = true;
        puedeVotar[_miembro] = _puedeVotar;
        _emitirTokensGob(_miembro, _tokensIniciales);
        emit MiembroRegistrado(_miembro);
    }

    function revocarMiembro(address _miembro) external soloAdmin {
        require(esMiembro[_miembro], "No es miembro");
        esMiembro[_miembro] = false;
        puedeVotar[_miembro] = false;
        emit MiembroRevocado(_miembro);
    }

    function actualizarDerechoVoto(address _miembro, bool _estado) external soloAdmin {
        require(esMiembro[_miembro], "No es miembro");
        puedeVotar[_miembro] = _estado;
        emit DerechoVotoActualizado(_miembro, _estado);
    }

    function _emitirTokensGob(address _destinatario, uint256 _cantidad) internal {
        saldoTokensGob[_destinatario] += _cantidad;
        suministroTotalGob += _cantidad;
        emit TokensGobEmitidos(_destinatario, _cantidad);
    }

    // -----
    // TESORERIA EN BLOCKCHAIN
    // -----

    event Deposito(address remitente, uint256 monto);
    event FondosEnviados(address destinatario, uint256 monto);

    receive() external payable {
        emit Deposito(msg.sender, msg.value);
    }

    function saldoTesoreria() public view returns (uint256) {
        return address(this).balance;
    }

    // -----
    // PROPUESTAS Y VOTACION (QUORUM 51%)
    // -----

```

```

struct Propuesta {
    uint256 id;
    address proponente;
    string descripcion;
    uint256 inicio;
    uint256 fin;
    uint256 votosAFavor;
    uint256 votosEnContra;
    bool ejecutada;
    bool cancelada;
    address payable billeteraProyecto;
    uint256 montoSolicitado;
}

```

```

uint256 public contadorPropuestas;
mapping(uint256 => Propuesta) public propuestas;
mapping(uint256 => mapping(address => bool)) public yaVoto;

```

```

event PropuestaCreada(
    uint256 id,
    address proponente,
    string descripcion,
    address billeteraProyecto,
    uint256 montoSolicitado,
    uint256 inicio,
    uint256 fin
);

```

```

event VotoEmitido(uint256 idPropuesta, address votante, bool aFavor, uint256 peso);
event PropuestaEjecutada(uint256 idPropuesta, address billeteraProyecto, uint256
monto);
event PropuestaCancelada(uint256 idPropuesta);

```

```

function crearPropuesta(
    string calldata _descripcion,
    address payable _billeteraProyecto,
    uint256 _montoSolicitado,
    uint256 _duracionSegundos
) external {
    require(esMiembro[msg.sender], "Solo miembros pueden proponer");
    require(_duracionSegundos > 0, "Duracion invalida");

    contadorPropuestas++;
    uint256 id = contadorPropuestas;

    propuestas[id] = Propuesta({
        id: id,
        proponente: msg.sender,

```



```

        descripcion: _descripcion,
        inicio: block.timestamp,
        fin: block.timestamp + _duracionSegundos,
        votosAFavor: 0,
        votosEnContra: 0,
        ejecutada: false,
        cancelada: false,
        billeteraProyecto: _billeteraProyecto,
        montoSolicitado: _montoSolicitado
    });

    emit PropuestaCreada(
        id,
        msg.sender,
        _descripcion,
        _billeteraProyecto,
        _montoSolicitado,
        block.timestamp,
        block.timestamp + _duracionSegundos
    );
}

function votar(uint256 _idPropuesta, bool _aFavor) external {
    Propuesta storage p = propuestas[_idPropuesta];
    require(p.inicio > 0, "Propuesta inexistente");
    require(puedeVotar[msg.sender], "No tiene derecho a voto");
    require(block.timestamp >= p.inicio, "Votacion no ha iniciado");
    require(block.timestamp <= p.fin, "Votacion finalizada");
    require(!yaVoto[_idPropuesta][msg.sender], "Ya voto");
    require(!p.cancelada, "Propuesta cancelada");

    uint256 peso = saldoTokensGob[msg.sender];
    require(peso > 0, "Sin tokens de gobernanza");

    yaVoto[_idPropuesta][msg.sender] = true;

    if (_aFavor) {
        p.votosAFavor += peso;
    } else {
        p.votosEnContra += peso;
    }

    emit VotoEmitido(_idPropuesta, msg.sender, _aFavor, peso);
}

function poderTotalVoto() public view returns (uint256) {
    return suministroTotalGob;
}

```

```

function tieneQuorum(uint256 _idPropuesta) public view returns (bool) {
    Propuesta storage p = propuestas[_idPropuesta];
    uint256 votosTotales = p.votosAFavor + p.votosEnContra;
    uint256 quorumRequerido = (poderTotalVoto() * 51) / 100;
    return votosTotales >= quorumRequerido;
}

function estaAprobada(uint256 _idPropuesta) public view returns (bool) {
    Propuesta storage p = propuestas[_idPropuesta];
    if (!tieneQuorum(_idPropuesta)) return false;
    return p.votosAFavor > p.votosEnContra;
}

function ejecutarPropuesta(uint256 _idPropuesta) external {
    Propuesta storage p = propuestas[_idPropuesta];
    require(!p.ejecutada, "Ya ejecutada");
    require(!p.cancelada, "Propuesta cancelada");
    require(block.timestamp > p.fin, "Votacion aun en curso");
    require(estaAprobada(_idPropuesta), "No aprobada o sin quorum");
    require(address(this).balance >= p.montoSolicitado, "Fondos insuficientes");

    p.ejecutada = true;
    (bool exito, ) = p.billeteraProyecto.call{value: p.montoSolicitado}("");
    require(exito, "Error al enviar fondos");

    emit PropuestaEjecutada(_idPropuesta, p.billeteraProyecto, p.montoSolicitado);
}

function cancelarPropuesta(uint256 _idPropuesta) external soloAdmin {
    Propuesta storage p = propuestas[_idPropuesta];
    require(!p.ejecutada, "Ya ejecutada");
    p.cancelada = true;
    emit PropuestaCancelada(_idPropuesta);
}

// -----
// PROGRAMA DE RECICLAJE E INCENTIVOS (OPERADO POR LA PAPAYA)
// -----

struct RegistroReciclaje {
    uint256 id;
    address reciclador;
    uint256 kilos;
    uint256 timestamp;
    uint256 tokensIncentivo;
}

```

```

uint256 public contadorReciclaje;
mapping(uint256 => RegistroReciclaje) public registrosReciclaje;

event ReciclajeRegistrado(
    uint256 id,
    address reciclador,
    uint256 kilos,
    uint256 tokensIncentivo
);

uint256 public factorIncentivo = 10; // tokens de gobernanza por kg reciclado

function actualizarFactorIncentivo(uint256 _nuevoFactor) external soloAdmin {
    factorIncentivo = _nuevoFactor;
}

// Solo La Papaya puede registrar reciclaje (operador territorial)
function registrarReciclaje(address _reciclador, uint256 _kilos) external soloLaPapaya {
    require(_reciclador != address(0), "Reciclador invalido");
    require(_kilos > 0, "Cantidad invalida");

    contadorReciclaje++;
    uint256 id = contadorReciclaje;

    uint256 tokens = _kilos * factorIncentivo;
    _emitirTokensGob(_reciclador, tokens);

    registrosReciclaje[id] = RegistroReciclaje({
        id: id,
        reciclador: _reciclador,
        kilos: _kilos,
        timestamp: block.timestamp,
        tokensIncentivo: tokens
    });

    emit ReciclajeRegistrado(id, _reciclador, _kilos, tokens);
}

// -----
// MODULO CLIMATICO: INDICADORES, UMBRALES Y CO2e
// -----

struct IndicadorAmbiental {
    uint256 id;
    string tipo; // "agua", "suelo", "movilidad", "residuos"
    string nombre; // "DBO", "SST", "OD", "ReciclajeKg", etc.
    uint256 valor;
    uint256 timestamp;
}

```

```

    string unidad;
}

uint256 public contadorIndicadores;
mapping(uint256 => IndicadorAmbiental) public indicadoresAmbientales;

event IndicadorAmbientalRegistrado(
    uint256 id,
    string tipo,
    string nombre,
    uint256 valor,
    string unidad,
    uint256 timestamp
);

struct Umbral {
    bool definido;
    uint256 minimo;
    uint256 maximo;
    bool menorEsMejor; // true: valor bajo es bueno (DBO, SST); false: valor alto es bueno
(OD, cobertura)
}

mapping(string => Umbral) public umbrales; // nombre -> umbral

event UmbralActualizado(
    string nombre,
    uint256 minimo,
    uint256 maximo,
    bool menorEsMejor
);

// Admin (con La Papaya en el comité) define umbrales según tablas de monitoreo
function actualizarUmbral(
    string calldata _nombre,
    uint256 _minimo,
    uint256 _maximo,
    bool _menorEsMejor
) external soloAdmin {
    umbrales[_nombre] = Umbral({
        definido: true,
        minimo: _minimo,
        maximo: _maximo,
        menorEsMejor: _menorEsMejor
    });

    emit UmbralActualizado(_nombre, _minimo, _maximo, _menorEsMejor);
}

```

```

// Oráculos ambientales (La Papaya, estudiantes, aliados) registran indicadores
function registrarIndicadorAmbiental(
    string calldata _tipo,
    string calldata _nombre,
    uint256 _valor,
    string calldata _unidad
) external soloOraculoAmbiental {
    require(_valor > 0, "Valor invalido");

    contadorIndicadores++;
    uint256 id = contadorIndicadores;

    indicadoresAmbientales[id] = IndicadorAmbiental({
        id: id,
        tipo: _tipo,
        nombre: _nombre,
        valor: _valor,
        timestamp: block.timestamp,
        unidad: _unidad
    });

    emit IndicadorAmbientalRegistrado(
        id,
        _tipo,
        _nombre,
        _valor,
        _unidad,
        block.timestamp
    );
}

function indicadorEnRangoBueno(uint256 _idIndicador) public view returns (bool
enRango, bool definido) {
    IndicadorAmbiental storage ind = indicadoresAmbientales[_idIndicador];
    Umbral storage umb = umbrales[ind.nombre];
    if (!umb.definido) {
        return (false, false);
    }

    if (umb.menorEsMejor) {
        enRango = (ind.valor <= umb.maximo);
    } else {
        enRango = (ind.valor >= umb.minimo);
    }
    return (enRango, true);
}

```

```

struct RegistroCO2 {
    uint256 id;
    string fuente; // "reciclaje", "huertas", "arboles", "movilidad", etc.
    uint256 co2e; // en kg (o t, segun definicion)
    uint256 timestamp;
    address beneficiario;
}

uint256 public contadorCO2;
mapping(uint256 => RegistroCO2) public registrosCO2;

event CO2Registrado(
    uint256 id,
    string fuente,
    uint256 co2e,
    address beneficiario,
    uint256 timestamp
);

// climateAdmin (con La Papaya y estudiantes detrás) registra CO2e ya calculado y
validado
function registrarCO2e(
    string calldata _fuente,
    uint256 _co2e,
    address _beneficiario
) external soloClimateAdmin {
    require(_beneficiario != address(0), "Beneficiario invalido");
    require(_co2e > 0, "CO2e invalido");

    contadorCO2++;
    uint256 id = contadorCO2;

    registrosCO2[id] = RegistroCO2({
        id: id,
        fuente: _fuente,
        co2e: _co2e,
        timestamp: block.timestamp,
        beneficiario: _beneficiario
    });

    emit CO2Registrado(id, _fuente, _co2e, _beneficiario, block.timestamp);

    _emitirRCC(_beneficiario, _co2e);
}

// -----
// TOKEN CLIMATICO: RIO CALI CARBON (RCC)
// -----

```

```

string public nombreRCC = "Rio Cali Carbon";
string public simboloRCC = "RCC";

mapping(address => uint256) public saldoRCC;
uint256 public suministroTotalRCC;

event RCCEmitido(address beneficiario, uint256 cantidad);

function _emitirRCC(address _beneficiario, uint256 _cantidad) internal {
    saldoRCC[_beneficiario] += _cantidad;
    suministroTotalRCC += _cantidad;
    emit RCCEmitido(_beneficiario, _cantidad);
}

// -----
// GESTION DE ROLES (INCLUYENDO ONG LA PAPAYA)
// -----

function cambiarLaPapaya(address _nuevaLaPapaya) external soloAdmin {
    laPapaya = _nuevaLaPapaya;
    esOraculoAmbiental[_nuevaLaPapaya] = true;
}

function transferirAdmin(address _nuevoAdmin) external soloAdmin {
    require(_nuevoAdmin != address(0), "Admin invalido");
    admin = _nuevoAdmin;
}

function actualizarClimateAdmin(address _nuevoClimateAdmin) external soloAdmin {
    require(_nuevoClimateAdmin != address(0), "ClimateAdmin invalido");
    climateAdmin = _nuevoClimateAdmin;
    esOraculoAmbiental[_nuevoClimateAdmin] = true;
}

function actualizarOraculoAmbiental(address _cuenta, bool _estado) external soloAdmin {
    esOraculoAmbiental[_cuenta] = _estado;
}
}

```